GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-11

今日榜单

1. **morluto/rea** — TypeScript
   Reverse engineer anything with agents, from app behavior down to native binaries... — 6天
2. **boykopovar/AnyPS5** — C++
   Tool for automatic PS5 executables porting to Linux and Windows — 7天
3. **storytold/artcraft** — Rust
   ArtCraft is an intentional crafting engine for artists, designers, and filmmaker... — 4天
4. **cathrynlavery/diagram-design** — HTML
   Editorial diagram design for Claude Code, Codex, GitHub Copilot, Cursor, Factory... — 5天
5. **bilawalsidhu/gods-eye-view** — JavaScript
   A spy satellite simulator in your browser, except the data is real. Live open so... — NEW
6. **pytorch/pytorch** — Python
   Tensors and Dynamic neural networks in Python with strong GPU acceleration — 2天
7. **lyogavin/airllm** — Jupyter Notebook
   AirLLM 70B inference with single 4GB GPU — NEW
8. **chrislgarry/Apollo-11** — Assembly
   Original Apollo 11 Guidance Computer (AGC) source code for the command and lunar... — NEW

#1 TypeScript 连续 6 天

rea

Reverse engineer anything with agents, from app behavior down to native binaries.

Stars **93,050** Forks **20,768** Today **+25,585**

https://github.com/morluto/rea

项目简介：REA 是一个基于 MCP（Model Context Protocol）的逆向工程工具集，让 AI Agent 能够分析原生二进制、JavaScript/Electron 应用、.NET 程序集和网站。它解决的核心问题是：当你看到一个应用里的功能想在自己的项目中复刻时，无需源代码即可理解其实现原理。所有分析在本地运行，并附带证据和局限性说明。

核心功能：一是通过 MCP 协议将逆向分析能力接入 Claude Code、Cursor、Codex 等主流 AI Agent；二是支持多类型目标分析，包括原生二进制、JS/Electron 应用、.NET 程序集和 Web 站点；三是可桥接 Hopper、Ghidra、IDA 等专业逆向工具进行原生分析；四是提供终端命令行接口，可独立于 Agent 使用；五是输出包含证据链和局限性的分析结论，而非黑盒答案。

适用场景：适合想从竞品应用中学习功能实现的产品开发者、需要快速理解闭源软件行为的安全研究人员，以及希望用 AI 辅助逆向工程流程的效率工程师。典型用法是让 Agent 分析某个 App 的特定功能（如搜索），理解其机制后在自己的项目中构建类似实现。

技术亮点：以 MCP 为协议层，将逆向工程工具链标准化为 Agent 可调用的工具目录，这是当前 AI Agent 生态中较为前沿的集成方式。静态 JavaScript 分析无需原生引擎即可运行，降低了使用门槛；原生分析则复用已有的 Hopper/Ghidra/IDA 安装，避免重复造轮子。TypeScript 实现且通过 npm 分发，安装体验流畅。

上手建议：最快的方式是运行 `npx rea-agents setup`，按提示选择你的 Agent 并批准配置变更，重启后即可用自然语言让 Agent 执行逆向分析。若只想在终端试用，可用 `npx -y rea-agents@latest analyze-javascript-application <路径> --json` 分析本地 JS/Electron 应用。想贡献代码可从 GitHub 仓库的 CI 流程和 MCP 工具契约文档入手，社区 Discord 也可获取支持。

#2 C++ 连续 7 天

AnyPS5

Tool for automatic PS5 executables porting to Linux and Windows

Stars **28,910** Forks **2,469** Today **+3,977**

https://github.com/boykopovar/AnyPS5

项目简介：AnyPS5 是一个用 C++ 编写的工具，目标是把 PS5 可执行文件自动移植到 Linux 和 Windows 平台。它通过重链接器把可执行文件转换为目标系统的原生格式，并实现 PS5 系统 prx 库以支持动态链接，从而在不依赖模拟器或独立运行时进程的情况下运行游戏。

核心功能：一是重链接器（relinker），将 PS5 可执行文件转换为目标平台原生格式；二是系统 prx 库实现，为动态链接提供兼容层；三是着色器重编译器，可将 PS5 着色器转换为 SPIR-V 并支持 Spirv-Tools 校验；四是输入映射，支持 SDL 手柄以及键盘鼠标的 ini 配置；五是兼容性追踪，通过徽章和进度图展示库与着色器的覆盖情况。

适用场景：主要面向希望在现代 PC 上运行 PS5 游戏的玩家，以及研究主机二进制格式、系统库和着色器移植的技术人员。也适合游戏保存与互操作性研究场景，例如已验证的《Dreaming Sarah》可在 GTX 1050 Ti / i5-7500 上稳定 60 fps 运行。

技术亮点：项目明确拒绝模拟器路线，转而采用原生格式转换加动态链接的方式，理论上能获得更接近原生的性能。着色器重编译器直接产出经 Spirv-Tools 验证的 SPIR-V，说明图形管线走的是 Vulkan 生态。异常处理上采用严格策略，遇到不支持状态直接抛出 `std::runtime_error` 并终止进程，便于快速暴露问题。

上手建议：先阅读 `docs/user/USAGE.md` 了解基本用法，再查看 `docs/user/COMPATIBILITY.md` 确认自己想跑的游戏是否在验证列表中。想参与贡献的话，建议从 `docs/dev/BUILD.md` 和 `docs/dev/ARCHITECTURE.md` 入手，同时注意项目有明确的代码风格约定和技术债文档，提交前可先熟悉 `CONTRIBUTING.md`。

#3 Rust 连续 4 天

artcraft

ArtCraft is an intentional crafting engine for artists, designers, and filmmakers

Stars **16,607** Forks **2,701** Today **+2,956**

https://github.com/storytold/artcraft

项目简介：ArtCraft 是一款面向艺术家、设计师和电影制作人的“AI 创作 IDE”，由 Rust 编写。它试图解决当前 AI 图像/视频生成工具“只能靠提示词碰运气”的痛点，把生成过程从“抽卡式提示”转变为可精确控制、可复现的“手工创作”。

核心功能：2D/3D 图像合成（图层、背景移除、景深布局）；Image to Location（在同一虚拟环境中规划多镜头）；Image to 3D Mesh（把图片转为可旋转摆放的 3D 物体）；角色摆姿与相机调度（生成前先摆好人物和机位）；Kitbashing 场景搭景（用 3D 素材拼装控制机位与空间关系）。

适用场景：适合分镜师、概念艺术家、独立电影人和内容创作者，在需要保持角色一致性、场景连贯性和镜头可控性的项目中使用——比如为同一房间规划多个镜头、为角色设定固定形象后批量出图，或搭建有纵深感的合成画面。

技术亮点：用 Rust 构建桌面级 IDE，性能和响应速度值得期待；把传统 3D 内容管线（建模、摆姿、机位、合成）与生成式模型结合，强调“先搭场景再生成”的工作流；模型可自选，避免绑定单一供应商，且强调结果的可重复性。

上手建议：普通用户可直接从 getartcraft.com 下载成品，或看官方两分钟 YouTube 演示快速了解工作流；开发者可参考仓库中的 `_docs/dev_setup.md` 从源码构建，并通过 Discord 社区交流。想贡献代码的话，建议先从 issues 和文档入手，熟悉其 Rust 架构与 3D/2D 合成模块。

#4 HTML 连续 5 天

diagram-design

Editorial diagram design for Claude Code, Codex, GitHub Copilot, Cursor, Factory Droid, and Pi. 44 diagram types. Self-contained HTML + SVG. No shadows. No Mermaid slop.

Stars **49,859** Forks **3,161** Today **+1,083**

https://github.com/cathrynlavery/diagram-design

项目简介：diagram-design 是一个面向 AI 编程助手（Claude Code、Cursor、Codex、Copilot 等）的“技能包”，让 Agent 用自然语言直接生成排版考究的编辑级图表。它解决的是 Mermaid 类工具生成的图表“能用但难看”的问题——输出的是单文件 HTML+SVG，可双击打开、可嵌入任何地方。

核心功能：内置 44 种图表类型（架构图、象限图、时序图等），Agent 会自行判断该用哪种；支持品牌定制，通过“Onboard diagram-design 到你的网站”自动抓取配色和字体，让所有图表风格统一；支持从 draw.io、Mermaid、Excalidraw 导入，并可导出 PNG/SVG；通过 `npx skills add` 一行命令跨多个 Agent 宿主安装，Claude Code 还提供插件市场和自动更新。

适用场景：适合需要频繁产出架构图、流程图、象限分析的技术团队和独立开发者，尤其是对视觉品质有要求、又不想手动拖拽画图的场景。也适合咨询、产品、设计等需要“给老板/客户看”的正式图表的角色。

技术亮点：输出完全自包含的 HTML+SVG，无外部依赖、无阴影、无 Mermaid 式“流水线感”，强调编辑级排版美学；作为 Agent Skill 遵循跨宿主标准（SKILL.md + references/assets/scripts），用 symlink 机制实现一份安装多处生效；README 明确列出网络请求范围（PRIVACY.md），并声明官方构建仅来自本仓库，对供应链安全有意识。

上手建议：最快的方式是运行 `npx skills add cathrynlavery/diagram-design`，然后在 Agent 里用大白话描述需求即可。想深度使用可先跑一次“onboard 到我的网站”让图表匹配品牌，再参考 Gallery 里的 44 种类型挑选。贡献方面，项目有 CI 和 CHANGELOG，建议先从新增图表类型或改进现有模板入手，遵循仓库的 Agent Skill 结构提交 PR。

#5 JavaScript NEW

gods-eye-view

A spy satellite simulator in your browser, except the data is real. Live open source spatial intelligence on a photorealistic 3D globe.

Stars **51,043** Forks **10,283** Today **+872**

https://github.com/bilawalsidhu/gods-eye-view

项目简介：God's Eye View 是一个运行在浏览器里的“间谍卫星模拟器”，但它展示的并非虚构数据，而是来自公开数据源的实时空间情报——航班应答机、船舶信标、卫星轨道、地震监测、公共摄像头等。它把这些散落的公开信号汇聚到一个逼真的 3D 地球上，让普通人也能像坐在指挥舱里一样观察真实世界。

核心功能：一是“驾驶舱视角”，可钻入任意被追踪的航班内部，镜头随地形起伏；二是“点击追踪”，锁定飞机、船只甚至火灾后自动绘制轨迹并展示元数据，还能一键切换到最近的实时摄像头；三是多种传感器滤镜（CRT、夜视、热成像、黑白等）和军用 HUD 叠加层，营造出强烈的战术氛围；四是语音控制与语音白板，可用自然语言提问（如某卫星何时过境）或在地球上直接标注路线和区域；五是天气与卫星过境预测，支持风场动画、雷达回放、气旋路径追踪等。

适用场景：适合开源情报（OSINT）爱好者、地理信息可视化研究者、航空/航海追踪迷，以及做数据新闻或演示的创作者。它也被用于 YouTube 视频系列的内容制作，既能作为教学工具展示公开数据的价值，也能作为技术 Demo 展示 3D 渲染与实时数据融合的能力。

技术亮点：项目用 JavaScript 在浏览器端实现逼真的 3D 地球渲染，并将多种异构实时数据流（ADS-B、AIS、TLE、地震、天气）统一到同一时空框架下。值得关注的是其模块化架构——每个数据层都是独立模块，便于扩展；同时支持无需 API Key 即可启动，降低了上手门槛。语音交互背后接入了实时 AI Agent，把自然语言查询转化为对地球数据的操作。

上手建议：README 明确建议“无需 API Key 即可启动”，可通过 Pinokio 一键安装或直接在终端本地运行，因此第一步是跑起来看效果。之后可以按“First Five Minutes”指南体验核心交互，再阅读“Under the Hood”了解数据层结构。若想贡献，可从自己熟悉的数据源入手，按模块化方式新增一个图层，并参考 CONTRIBUTING.md 的流程提交。

#6 Python 连续 2 天

pytorch

Tensors and Dynamic neural networks in Python with strong GPU acceleration

Stars **104,345** Forks **32,064** Today **+314**

https://github.com/pytorch/pytorch

项目简介：PyTorch 是由 Meta（原 Facebook）主导开发的开源深度学习框架，目前由 Linux 基金会下的 PyTorch 基金会管理。它提供了类似 NumPy 的张量计算能力并原生支持 GPU 加速，同时内置基于磁带式自动微分（tape-based autograd）的动态神经网络系统。它解决的核心问题是：让研究人员和工程师能以接近 Python 原生编程的体验，灵活、高效地构建和训练深度学习模型。

核心功能： - **GPU 就绪的张量库（torch）**：提供切片、索引、线性代数、归约等丰富算子，可在 CPU/GPU 间无缝切换，性能强劲。 - **动态自动微分（torch.autograd）**：采用反向模式自动微分与磁带机制，支持运行时动态改变网络结构，调试直观、灵活性极高。 - **神经网络模块（torch.nn）**：与 autograd 深度集成，提供层、损失函数、优化器等组件，兼顾灵活性与开发效率。 - **多进程与数据加载（torch.multiprocessing / torch.utils）**：支持跨进程共享张量内存，方便数据加载与 Hogwild 式并行训练。

适用场景：主要面向深度学习研究者和工程师，用于学术研究中的模型原型验证、计算机视觉/NLP/强化学习等领域的模型训练，以及工业界的生产级模型部署。也常被数据科学家当作 GPU 版 NumPy 用于通用科学计算。由于其 Python 优先的设计，熟悉 NumPy/SciPy 的用户几乎可以零成本迁移。

技术亮点：最值得关注的是其"磁带式"动态计算图——与 TensorFlow 早期静态图不同，PyTorch 在前向传播时实时记录操作，反向传播时回放，使得网络行为可任意改变而无延迟或额外开销。此外，"Python First"理念让框架深度融入 Python 生态，可直接复用 NumPy、SciPy、Cython 等包进行扩展，这也是它迅速取代早期框架、成为学术界主流的关键原因。

上手建议：新手建议从官方教程（pytorch.org/tutorials 的 "Learn the basics"）入手，通过 `pip install torch` 或 conda 安装二进制包即可快速体验。想深入源码或贡献代码，可先阅读 README 中的 "From Source" 编译指南，并关注 hud.pytorch.org 上的 CI 健康状态；社区交流主要通过 GitHub Issues、论坛和 Slack 进行，建议从文档改进或标注为 "good first issue" 的任务开始参与。

#7 Jupyter Notebook NEW

airllm

AirLLM 70B inference with single 4GB GPU

Stars **36,235** Forks **3,799** Today **+541**

https://github.com/lyogavin/airllm

项目简介：AirLLM 是一个让超大语言模型在极小显存下运行的开源推理框架，核心卖点是无需量化、蒸馏或剪枝，就能用单张 4GB 显卡跑 70B 模型。它通过分层流式加载的方式，把原本需要多卡才能装下的模型"拆"进有限的显存里。

核心功能：一是极致省显存推理，宣称可在 4GB 以下跑 Kimi K3（2.8T）、6GB 跑 Qwen3.8-Flash-Next（125B）、约 12GB 跑 DeepSeek-V3（671B）；二是统一 `AutoModel` 接口，自动识别模型类型，覆盖 Qwen、Llama、DeepSeek、Phi、Gemma 等主流模型；三是支持 8bit/4bit 量化、CPU 推理、safetensors 及非分片模型；四是新增训练支持，通过逐层流式加载冻结权重、把适配器留在 GPU 上，实现小显存微调大模型。

适用场景：主要面向显存有限的个人开发者、学生和小型团队，让他们在消费级显卡（如 RTX 3060 Ti/3090/4090）上体验或调试百亿、千亿参数模型。也适合做本地化部署验证、模型效果评估，以及资源受限环境下的 LoRA 微调实验。

技术亮点：核心思路是"分层流式推理"——不把整个模型一次性载入显存，而是按层加载、用完即释放，并配合预取（prefetching）让加载与计算重叠以提升速度。针对 MoE 模型还做了按专家路由的流式加载，只加载 token 实际用到的专家，这是它能把 2.8T 模型压到 3.72GB 的关键。

上手建议：从 README 的 Quickstart 和示例 notebook 入手，先跑通一个小模型（如 Llama3 70B 或 Qwen）验证环境。注意它的性能瓶颈在磁盘 I/O 和内存带宽，建议配足内存（如 64GB）和高速 SSD。想贡献可从支持新模型架构、优化预取策略或补充文档入手，项目用 Jupyter Notebook 编写，便于快速实验。

#8 Assembly NEW

Apollo-11

Original Apollo 11 Guidance Computer (AGC) source code for the command and lunar modules.

Stars **73,990** Forks **8,088** Today **+496**

https://github.com/chrislgarry/Apollo-11

项目简介：这是阿波罗11号制导计算机（AGC）的原始汇编源码仓库，包含指令舱（Comanche055）和登月舱（Luminary099）两套程序。它并非一个可运行的现代软件项目，而是一份数字化的历史技术档案，解决了原始代码濒临失传、公众难以接触的问题。

核心功能：保存并公开阿波罗11号指令舱与登月舱的完整AGC源码；提供与MIT博物馆原始扫描件的对照勘误机制；支持通过Virtual AGC工具链对源码进行汇编；附带多语言README翻译，降低全球开发者的阅读门槛。

适用场景：航天史研究者、嵌入式与汇编语言爱好者、计算机科学教育者，以及任何对“人类如何用64KB内存登月”感到好奇的开发者。常用于技术史教学、复古计算研究或软件考古。

技术亮点：代码为1969年MIT仪器实验室手写的汇编语言，运行于约2MHz、内存仅约4KB的AGC上，却实现了导航、制导与控制等关键任务。仓库还保留了原始合同审批页和程序版本信息，具有极高的史料价值。

上手建议：若只想阅读，直接浏览`Comanche055/`和`Luminary099/`目录即可；若想编译运行，需先了解Virtual AGC项目。贡献前请阅读CONTRIBUTING.md，主要工作是比对源码与原始扫描件、提交转录错误修正。

数据来源：github.com/trending

报告日期：2026-10-11

由 DeepSeek AI 提供智能分析 | 自动生成